

Project Overview The goal of this project is to architect a highly secure, human-in-the-loop (HITL) AI assistant that allows users to interact with Dremio data catalogs. By leveraging LangGraph for state management and cyclic routing, alongside the Dremio Model Context Protocol (MCP) server, we will create a chatbot that autonomously explores metadata, drafts SQL, and executes queries while strictly enforcing data governance protocols. This architecture relies on three main components: an LLM Layer for reasoning, an MCP Client/Host for managing communication, and the Dremio MCP Server to provide standardized data tools 1.

1. System Architecture & Logic Graph

State Management & Human-In-The-Loop (HITL) To satisfy the iterative refinement and metadata validation steps, the system utilizes LangGraph's StateGraph combined with a MemorySaver checkpoint 2, 3. This enables the graph to pause execution (interrupt_before) and save its current state, allowing the user to review, authorize, or correct the agent's trajectory before it proceeds 4, 5. The state is managed via a TypedDict containing the conversation history (Annotated[list, add_messages]), the current drafted SQL, and the targeted schemas 6.

LangGraph Node Descriptions

- **guardrail_node**: Acts as the entry point. It evaluates the user's intent to ensure the query is domain-specific (data/analytics) and safely read-only. While the Dremio MCP RunSqlQuery tool inherently blocks DML/DDL statements (like INSERT, UPDATE, DROP) unless explicitly configured otherwise 7, 8, this node serves as the first line of defense to intercept malicious intents early.
- **discovery_node**: Interfaces with the MCP Client to execute tools like SearchTableAndViews (for semantic search of matching tables) and GetUsefulSystemTableNames to locate the correct data sources 9, 10. It then invokes GetSchemaOfTable to pull exact column names and data types 11.
- **metadata_confirmation_node**: Formats the discovered schemas and tables, then halts graph execution using an interrupt. It waits for the user to validate the sources before proceeding 4, 12.
- **sql_gen_node**: An LLM call that takes the validated schema and the user's original question to generate a precise, read-only SQL query.
- **refinement_node**: Presents the drafted SQL to the user. This node accepts human feedback to rewrite the query. It utilizes a cyclic graph structure (a tools_condition or custom routing loop) to continually pass feedback back to the sql_gen_node until the user provides an explicit "Proceed" signal 13, 14.
- **execute_node**: Invokes the RunSqlQuery tool via MCP to execute the approved query against the Dremio cluster and formats the resulting data into a user-friendly response 7.

State Machine Logic Graph (Mermaid.js)

```
graph TD
    %% Node Definitions
    START((START))
    Guardrail[guardrail_node: Intent & Safety Filter]
    Discovery[discovery_node: MCP Schema Search]
    MetaConfirm[metadata_confirmation_node: HITL Pause]
    SQLGen[sql_gen_node: Draft SQL]
```

```

Refinement[refinement_node: HITL SQL Review]
Execute[execute_node: MCP RunSqlQuery]
END_NODE((END))

%% Edge Connections
START --> Guardrail
Guardrail -- "Safe & In-Domain" --> Discovery
Guardrail -- "Unsafe/Out-of-Domain" --> END_NODE

Discovery --> MetaConfirm
MetaConfirm -- "User Approves Schema" --> SQLGen
MetaConfirm -- "User Rejects/Modifies" --> Discovery

SQLGen --> Refinement
Refinement -- "Provides Feedback" --> SQLGen
Refinement -- "Proceed Signal" --> Execute

Execute --> END_NODE

```

2. Project Structure & File Map

To ensure maintainability and separation of concerns, the repository is structured following best practices for LangGraph deployments 15.

Directory Tree

```

dremio-sql-agent/
├── .env
├── langgraph.json
├── requirements.txt
└── src/
    ├── __init__.py
    ├── agent.py
    ├── state.py
    ├── nodes.py
    ├── tools.py
    └── mcp_client.py

```

File Breakdown & Responsibilities

- **langgraph.json:** The configuration file required for LangGraph deployment, pointing to agent.py:graph and defining environment dependencies 15, 16.
- **src/state.py:** Defines the AgentState schema using Python's TypedDict. It includes fields for messages (using Annotated[list, add_messages]), discovered_schemas, drafted_sql, and approval_status 6, 17.
- **src/mcp_client.py:** Manages the initialization of the MCP Client/Host. It establishes the secure connection to the Dremio MCP Server, validates access, and handles tool discovery 18, 19.
- **src/tools.py:** Wraps the discovered Dremio MCP tools into LangChain-compatible @tool definitions. Specifically, it exposes SearchTableAndViews, GetSchemaOfTable, and RunSqlQuery to the LLM 20, 21.

- **src/nodes.py:** Contains the core business logic for each state machine step. It houses the functions for guardrail_node, discovery_node, sql_gen_node, and the HITL processing nodes.
- **src/agent.py:** The central orchestrator. It imports the state, nodes, and tools to construct the StateGraph. It sets the edges, binds the MemorySaver checkpoint for thread persistence, configures interrupt_before=["metadata_confirmation_node", "refinement_node"], and compiles the final graph 2, 5.

Import Structure & Interaction Graph (ASCII)

```
[mcp_client.py] <-----+
|                               |
v                               |
[tools.py] <-----+          |
|                   |         |
v                   |         |
[nodes.py] >-----+         |
|                   |         |
v                   |         |
[state.py] <----- [agent.py] -----+
|                               |
v                               |
                               (Compiled LangGraph)
```

- mcp_client.py establishes the host connection.
- tools.py consumes the client to map available capabilities.
- nodes.py imports tools.py to execute logic and state.py to read/write shared context.
- agent.py imports state.py, nodes.py, and mcp_client.py to wire the cyclic graph, assign persistence, and compile the final application.